

復旦大學

本科毕业论文



论文题目:

基于 RNN 和状态抽象的日志异常检测可解释性研究

姓 名: 谢森煜 学 号: 20307130140

院 系: 软件学院

专 业: 软件工程

指导教师: 彭鑫 职 称: 教授

单 位: 复旦大学软件学院

完成日期: 2023 年 5 月 15 日

论文撰写人承诺书

本毕业论文是本人在导师指导下独立完成的，内容真实、可靠。本人在撰写毕业论文过程中不存在请人代写、抄袭或者剽窃他人作品、伪造或者篡改数据以及其他学位论文作假行为。

本人清楚知道学位论文作假行为将会导致行为人受到不授予/撤销学位、开除学籍等处理（处分）决定。本人如果被查证在撰写本毕业论文过程中存在学位论文作假行为，愿意接受学校依法作出的处理（处分）决定。

承诺人签名：谢森煜

日期：2024 年 5 月 21 日

指导教师对论文学术规范的审查意见：

☐ 本人经过尽职审查，未发现毕业论文有学术不端行为。

☐ 本人经过尽职审查，发现毕业论文有如下学术不端行为：

指导教师签名：

日期： 20 年 月 日

指导教师评语：

答辩委员会（小组）评语：

签名：

20 年 月 日

签名：

20 年 月 日

学分

成绩

备注：

目录

摘要.....	II
ABSTRACT.....	III
第一章 引言	1
1.1 研究背景.....	1
1.2 研究目的和意义.....	1
1.3 核心内容和创新点.....	2
第二章 背景知识和研究现状.....	3
2.1 日志异常检测框架概述.....	3
2.1.1 日志收集.....	3
2.1.2 日志解析.....	4
2.1.3 特征提取.....	6
2.1.4 异常检测.....	7
2.2 基于机器学习的检测方法.....	8
2.3 基于深度学习的检测方法.....	10
2.4 基于状态抽象解释 RNN.....	11
第三章 模型构建与系统实现.....	13
3.1 工具概览.....	13
3.2 日志异常检测模型的实现.....	13
3.3 基于状态抽象的抽象模型构建.....	15
3.4 可视化系统概览.....	16
3.4.1 抽象状态图与训练集视图.....	17
3.4.2 在线预测与异常检测结果.....	18
第四章 实验验证与效果分析.....	20
4.1 日志异常检测效果.....	20
4.1.1 实验环境设置.....	20
4.1.2 测试结果展示.....	21
4.2 抽象模型效果.....	23
4.3 可视化系统使用场景.....	24
第五章 总结与展望.....	29
参考文献	30
致谢.....	33

摘要

随着计算机技术的发展和用户需求的增长，现代软件系统日益庞大，分布式系统开始流行，这一切增加了系统运维的难度。日志是监控系统运行的重要资源，能帮助运维人员检测异常、排除故障。然而，现代软件系统产生的海量日志使得传统的人工检查方法难以应对，因此，基于日志数据的自动化异常检测技术得到了迅速发展。

本毕业设计尝试基于 RNN 和状态抽象技术，探索日志异常检测的可解释性。在综述部分，本文阐述了现有的日志异常检测框架，包括日志收集、日志解析、特征提取和异常检测。虽然深度学习方法能够提高模型的适应性和数据处理能力，但也会令模型难以解释。因此文章引入状态抽象技术，通过构建抽象模型提升检测过程的可解释性。

本毕业设计的工作包括三个部分：首先，实现了一个用于训练日志异常检测模型的工具，允许用户自定义神经网络结构和输入日志序列形式。其次，实现了一个基于状态抽象的模型解释工具，通过聚类方法对 RNN 的隐向量进行抽象，生成抽象状态和状态轨迹。最后，构建了一个可视化工具，展示抽象模型和异常检测结果，以图表和状态图的形式呈现数据，并提供一定的交互性。

通过在真实日志数据上的实验验证，文章中所提的日志异常检测模型的性能得到了复现。利用状态抽象技术，基于 RNN 的异常检测模型被转换为抽象模型，并在输出上和原模型保持较高的一致性。最后，文章通过一个具体案例，结合可视化工具对一次异常检测过程进行了分析和解释，展示了用户可以探索模型的全局和局部行为并理解异常检测模型的推理。本毕业设计在智能运维和系统故障诊断的思路和方法上展开研究，尝试帮助用户更好地理解 and 诊断模型，从而提升异常检测的可信度和有效性。

关键词：异常检测，循环神经网络，日志数据分析，可视化，可解释人工智能

ABSTRACT

With the development of computer technology and the increasing demands of users, modern software systems are becoming more complex, and distributed systems are becoming prevalent, increasing the difficulty of system operations and maintenance. Logs are a critical resource for monitoring system operations, helping operations personnel detect anomalies and troubleshoot issues. However, the vast amount of logs generated by modern software systems makes traditional manual inspection methods impractical, leading to the rapid development of log-based anomaly detection techniques.

This thesis explores the interpretability of log anomaly detection based on RNN and state abstraction techniques. In the literature review, the paper introduces existing log anomaly detection frameworks, including log collection, log parsing, feature extraction, and anomaly detection. Although deep learning methods can enhance model adaptability and handle larger-scale data, they also make models harder to interpret. Therefore, the paper introduces state abstraction techniques to enhance the interpretability of the detection process by constructing abstract models.

The work of this thesis includes three parts: first, a tool for training log anomaly detection models was implemented, allowing users to customize neural network structures and input log sequence formats. Second, a model interpretation tool based on state abstraction was implemented, abstracting RNN hidden vectors using clustering methods to generate abstract states and state trajectories. Finally, a visualization tool was developed to display the abstract model and anomaly detection results, presenting data in the form of charts and state diagrams, and providing a certain degree of interactivity.

Through experiments on real log data, the performance of the log anomaly detection model proposed in the paper was reproduced. Using state abstraction techniques, the RNN-based anomaly detection model was converted into an abstract model, maintaining a high degree of

consistency with the original model's outputs. Finally, through a specific case study, the paper analyzed and explained an anomaly detection process using the visualization tool, demonstrating that users can explore the model's global and local behavior and understand the reasoning of the anomaly detection model. This thesis conducts research on the ideas and methods of intelligent operations and system fault diagnosis, aiming to help users better understand and diagnose models, thereby enhancing the credibility and effectiveness of anomaly detection.

Key words: anomaly detection, recurrent neural network, log data analysis, visualization, explainable AI

第一章 引言

1.1 研究背景

随着计算机技术的发展和用户需求的增长,现代软件系统逐渐庞大,同时为了提供高可用的服务,不少软件被设计成分布式系统。不同于传统的单体软件,分布式系统由多个节点组成,不同的节点之间交错形成复杂的依赖关系,故障可能表现为随机的、不可预测的行为,这一切极大地增加了运维的难度。

日志是软件系统运行过程中产生的各种记录,是监控系统运行的重要资源,能够帮助运维人员检测异常、排除故障。但是现代庞大的软件系统会产生海量日志,传统的人工检查已无法胜任,因此基于日志数据的自动化异常检测技术得到快速发展。

基于日志的异常检测是一种通过分析日志来监视系统、管理系统和检测故障的技术。RNN 是一种针对序列数据的神经网络模型,能处理文本、时间序列等。RNN 与传统的前馈神经网络有所不同,它具备记忆能力,能在处理序列数据时保留之前的信息。RNN 因其适用于处理序列数据,成为了一种常用于日志异常检测的深度学习模型,但是同时 RNN 的黑盒性也使其难以被理解和解释。

1.2 研究目的和意义

本文章旨在调研日志异常检测的整体流程和相关运维技术,并研究已有的基于 RNN 的日志异常检测模型,分析并验证这些方法在具体场景下的表现。

同时,本文章还调研了目前提出的一些 RNN 解释方法,尝试将 RNN 解释方法应用到基于 RNN 的日志异常检测模型中,为用户提供一定的模型解释,帮助用户理解、诊断模型,让基于日志的异常检测取得更好的效果。

在可视化数据的基础上,本毕业设计为用户提供一个可交互的界面,将模型整体解释、单个样本解释、异常检测过程呈现出来,降低日志异常检测、RNN 模型解释等技术的使用门槛,提升用户友好性。

当前人们对于深度学习模型的信任度不高,特别是在安全敏感领域。根本原因之一就是神经网络的可解释性存疑。本文章尝试将 RNN 模型的解释与日志异常检测结合,让检测结果具有一定的可解释性,让运维人员理解日志序列被标记为异常的原因,提高判断的可信度。

1.3 核心内容和创新点

本毕业设计将围绕日志异常检测流程展开，设计并实现一个完整的系统。该系统包括三个部分，分别能为用户提供模型训练、模型抽象、界面展示的功能。

第一个部分，用于训练日志异常检测模型的训练工具。该工具允许用户通过扩展代码和修改配置项的方式，自定义神经网络结构、输入日志序列形式、模型预测类型等。针对日志异常检测任务，提供了在测试集上的检测工具，以贴合实际的方式测试模型。

第二个部分，基于 RNN 和状态抽象的模型解释工具。该工具允许用户指定在第一个部分中训练好的模型，利用模型和对应训练集提取 RNN 的隐状态，并使用聚类等方法对收集的隐状态进行状态抽象，生成抽象路径、抽象状态描述等产物。

第三个部分，展示抽象模型和异常检测的可视化工具。在前两个部分的基础上，该工具以传统前后端的方式提供服务，后端负责运行模型、生成异常检测结果、返回训练集数据等，前端负责以图表的方式展示数据、以图结构的方式展示抽象模型等。同时，工具提供了一定的可交互性，允许用户在线进行模型推理。

本毕业设计尝试性地将基于 RNN 的日志异常检测和 RNN 模型的解释方法进行结合。一方面，利用 RNN 的解释方法帮助用户理解模型在异常检测过程中的内部行为，理解异常检测结果的产生。另一方面，RNN 的解释方法也有助于用户改进模型，提高模型精度。最终以可视化、可交互的方式呈现出多种视图，从不同粒度对基于 RNN 的日志异常检测进行解释。

第二章 背景知识和研究现状

在现代大型软件系统的运维中，异常检测扮演着重要的角色。很多大型软件系统肩负着提供高可用服务的使命，一次系统异常很可能会导致重大的经济损失。而异常检测负责在日常运维的过程中，及时检测系统是否发生异常，以尽早提醒运维人员和开发人员修复故障、恢复系统正常运行，降低经济损失。

而日志是现代软件系统产生的运行时记录，其记录着系统运行期间的各种行为信息。这种日志早已被作为软件系统异常检测的主要数据源[1]。传统情况下，运维人员往往需要人工检查日志，或者基于一些规则匹配来检测异常。然而现代庞大的软件系统每天都会产生海量日志，传统的检测方式已无法胜任。Shilin He 等人认为[1]：一方面，现代软件系统往往是分布式的、并行的，复杂的系统是对运维人员心智的巨大挑战；另一方面，现代软件系统产生海量日志，充斥噪声数据，搜索和匹配的效率低下；最后，软件系统有时会有不同的行为机制，传统方法会导致很多与实际异常无关的误报[3]。

为了降低人力成本，许多基于日志数据的自动异常检测技术被提出。基于日志的异常检测是一系列通过分析日志数据来监视系统、管理系统和检测故障的技术。日志异常检测是 AIOps (Artificial Intelligence for IT Operations)，即智能化运维，中十分重要的一部分，在过去几十年中得到了广泛的研究。

2.1 日志异常检测框架概述

来自香港中文大学的 LOGPAI 团队在其文章[1][2]中提到，日志异常检测的基本框架包括四个阶段：日志收集、日志解析、特征提取和异常检测。四个阶段如图 1 所示。

2.1.1 日志收集

软件系统一段时间内产生的日志将会被收集，然后经过一定的预处理并保存，以备后续使用。日志通常会包含时间戳和详细说明，详细说明则根据打印语句的不同有不同的格式和参数。

目前被广泛使用的 HDFS-v1 数据集[4]，正是通过收集来自 200 多个 Amazon EC2 节点上的 HDFS 日志得到的，并且由 Hadoop 领域的专家进行了标注。

HDFS-v1 数据集被收录在 LOGPAI 团队发布的 Loghub[7] 项目中。该项目维护

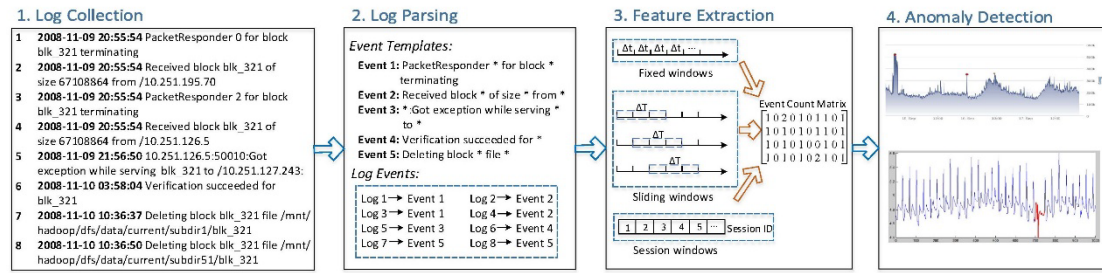


图 1: 日志异常检测基本框架[1]

了 19 组日志数据集, 这些数据集收集自分布式系统、超级计算机、操作系统、移动系统、服务器应用和独立软件, 总计达到 77GB 大小。该项目为日志数据分析带来巨大的便利, 从发布至今, 已经被工业界和学术界的组织下载了数万次。

在日志收集的技术层面, ELK 是一种常见的技术栈。ELK 是由 Elasticsearch、Logstash 和 Kibana 三者组成的: Elasticsearch 是一个分布式搜索存储引擎, 能提供高效的搜索和分析能力; Logstash 是一个数据收集引擎, 负责数据的收集和传输; Kibana 是一个数据可视化分析工具, 能为用户提供一系列直观的图表展示。

2.1.2 日志解析

日志是一种由固定模板内容和可变参数构成的纯文本。将日志转化为结构化的形式, 即分离出静态的模板部分和动态的参数列表, 这一过程就被称为日志解析, 如图 2 所示。日志解析的主要目的就是海量的日志中提取出一组日志事件模板, 并将各个日志转化为结构化的形式, 方便后续的特征提取。

例如 “Received block blk_561725380853097689 of size 66107865 from /10.261.92.74” 这条日志应当被解析为事件模板 “Received block <*> of size <*> from /<*>” 和参数列表 [“blk_561725380853097689”, “66107865”, “10.261.92.74”], 其中 “<*>” 是三个参数的占位符。而符合这种模板的日志可能还有很多, 它们都基于 “Received block <*> of size <*> from /<*>” 这一日志事件模板, 被解析后也都会被标记上该日志事件模板对应的编号。

传统的日志解析方式主要依赖手工编写正则表达式提取日志事件模板, 这种方法虽然简单通用, 但是需要人工阅读大量的日志, 对运维人员带来很大的心智负担, 容易出现编写错误。而且, 现代软件系统的版本频繁更迭, 其中的日志输出语句往往也会随之更新, 人工解析非常耗时耗力难以跟上版本更迭[9]。

为减少人工成本, 此前也有相关研究[4]尝试通过对源代码进行静态分析来提取日志事件模板。然而, 实际应用中源代码往往是难以获取的, 而且对不同平

台不同语言的软件构建静态分析也需要花费不少的成本。因此，数据驱动的方法成为自动化日志解析的主要选择。

日志解析有两种主要思路：基于聚类 and 基于启发式。基于聚类的方法首先计算日志之间的距离，然后用聚类算法将日志分簇，最后从每个簇中提取日志事件模板。基于启发式的方法先计算每个单词在各日志位置上的出现次数，接着挑选频繁出现的单词组合为事件候选项，最后选择一些候选项作为日志事件。

LOGPAI 团队发布的 Logparser 开源工具库实现了一系列近年来常见的日志解析器。在相关文章[8][9]中，研究人员基于 Logparser 在涵盖分布式系统、操作系统等领域的 16 个日志数据集上对各种日志解析器进行了评估。通过对比正确率、鲁棒性等指标，用户能更好地选择合适的日志解析器。

LogMine[10]是基于聚类的方法的代表，能通过层次聚类的方式生成事件模板，自底向上地将日志消息分组到簇中。该方法的优点是快速高效，并且有工业化的应用。SLCT[11]是最早使用启发式方法进行日志解析的开源工具，后续又有一些相关研究对这种方法进行了改进，但整体思路仍然延续频繁模式挖掘。

Drain[5]是一种流行的日志解析方法，它能以在线的方式高效地从原始日志数据中提取模板。读入一条日志，Drain 首先通过简单的正则表达式提取出其时间戳、级别等，然后用日志正文构建一棵具有固定深度的解析树，将具体的解析规则编码在解析树的节点中。

DivLog[6]是一种利用大语言模型上下文学习能力的方法。其预先选取适当的日志作为提示，让大模型通过挖掘提示中示例日志的语义，实现以无需训练的方式生成目标日志模板。

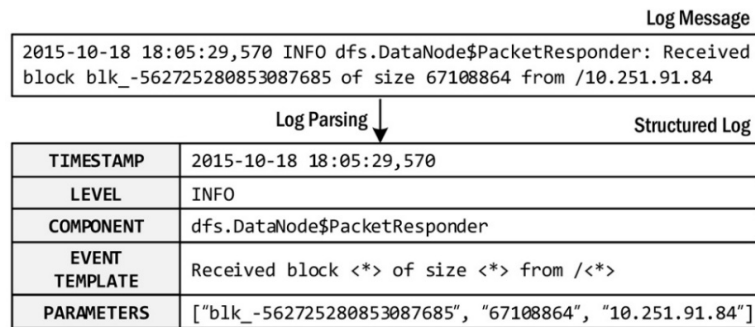


图 2：日志解析示例[8]

日志事件模板是从大量的日志中挖掘出来的，对于某个固定版本的软件系统，其日志事件模板的种类也是固定的。因此在日志解析后可以基于解析的结果，对数据集进行一定的处理，方便后续特征提取。

通常日志会先被标记出其对应的日志事件模板，进一步可以用一个 ID 来指代模板。例如 “Received block blk_561725380853097689 of size 66107865

from /10.261.92.74”这条日志对应的日志事件模板是“Received block <*> of size <*> from /<*>”，将这个模板编号为 E9，那么就可以用 E9 来代替原本的日志文本。本毕业设计所使用的 HDFS-v1[4]数据集，日志的原始文本文件的大小为 1.47GB，而利用解析结果将日志序列以日志模板编号序列的形式保存下来，文件的大小就被压缩为 119MB。例如，关于块 blk_2942936127925517624 的一系列日志所构成的日志序列，使用日志模板编号简化，就可以用 [E5, E22, E5, E7] 来表示。如果需要具体语义信息，只需要在日志事件模板表中，根据日志模板编号获取对应的完整模板文本即可。

2.1.3 特征提取

日志在被解析为结构化的形式后，无论是日志事件模板还是参数列表，都仍然是文本数据，它们需要被转化为数值特征，这样异常检测模型才可以使用这些数据进行训练、检测。

首先日志需要被划分为不同的组，同一组内的日志按照时间戳顺序排列为一个序列，称为日志序列。划分日志序列的过程通常需要基于时间戳或者日志的特殊 ID（例如 block ID），结合窗口划分的方式，日志数据可以被分为有限个相对独立的日志窗口。常见的窗口划分包括：固定窗口、滑动窗口和会话窗口[1]。

固定窗口：按照预先确定的窗口长度，将按时间戳排序后的日志分割成一系列固定长度的窗口。每个固定窗口内的日志可视为一个单独的序列，两两窗口之间不存在重叠的部分。一个固定窗口的长度一般代表窗口内的日志个数，有时也代表时间跨度。

滑动窗口：在确定窗口长度和滑动步长的基础上，先将日志按照时间戳排序，然后从排序后的日志的开头开始滑动，每次滑动固定的步长，形成一系列重叠的窗口序列。例如可以规定窗口长度为 30 分钟，每次滑动 5 分钟的长度。一般来说滑动步长会小于窗口长度，这也造成了序列之间存在重叠。

会话窗口：根据日志中特定的 ID 将一系列日志组合成一个序列，这些日志都含有某个 ID，表明它们产生自同一个任务的执行过程[2]。例如，HDFS 日志会记录对某一个 block 的分配、读写、删除等，因此可以根据 block ID 将一系列日志聚集并排序，形成一个会话窗口序列，每个会话窗口都与某个 block 的任务对应。此外，这样构建的日志序列往往因为具体任务不同而长短不一。

在日志划分之后许多基于机器学习的方法会生成日志事件计数向量：为每个序列构建计数向量，维度对应日志事件编号，值对应序列中该类型日志事件出现的次数。例如，对于计数向量 c ， c_9 就代表 E9 日志事件模板出现的次数。多个日志事件计数向量拼接在一起，组成一个日志事件计数矩阵 A ， a_{ij} 就代表日志事件

模板 E_j 在第 i 个日志序列中出现的次数。

不同于基于机器学习的方法, 基于深度学习的方法通常直接输入日志特征序列。日志特征序列中的一个元素可以直接是日志事件的编号, 也可以是更复杂的特征。例如对于一个序列 $[E5, E5, E22, E5, E11, E9, E11]$, 其对应的特征序列可以是 $[5, 5, 22, 5, 11, 9, 11]$, 也可以是 $[a_1, a_2, a_3, a_4, a_5, a_6, a_7]$, 其中每一个元素 a_i 是带有语义的日志嵌入向量。常见的日志嵌入向量的一种生成方法是, 先对日志事件模板中的单词生成词嵌入向量, 然后将词嵌入向量加权聚合, 得到日志事件模板的嵌入向量[12]。

2.1.4 异常检测

上一阶段提取的特征可以被用于模型训练, 训练好后的模型可以用于异常检测。基于机器学习的方法通常针对整个输入的日志序列判断是否异常, 因为这些方法都基于日志事件计数矩阵, 难以利用更细粒度的特征。而基于深度学习的方法通常直接使用日志特征序列训练, 学习正常的日志模式, 因此能够定位污染日志事件序列的确切日志事件, 提高了可解释性[2]。

本次毕业设计中使用到的日志异常检测模型主要是 Min Du 等人提出的 DeepLog[17], 其神经网络结构简单, 易于构建和分析, 但同时效果又十分优秀。并且 DeepLog 是一种基于序列预测的无监督学习方法, 不仅具有更细粒度的异常检测结果, 而且在大型数据集 (如 HDFS-v1) 中所需的训练数据仅占整体的很小一部分 (1%)。除此之外, 由于 DeepLog 是数据驱动模型, 随着软件系统版本的迭代, 即使日志模式发生了变化, DeepLog 也可以使用新的数据增量式地学习, 随着时间推移不断适应。

要利用神经网络进行日志异常检测, 需要确定合理的神经网络结构及其损失函数。DeepLog 受到自然语言处理的启发: 日志序列是遵循一定模式和语法规则的一种序列。实际上, 日志正是由遵循逻辑和控制流的程序产生的, 非常类似于自然语言。而异常发生时, 这种正常的日志模式往往会被破坏。例如, 由于异常的发生, 日志序列中的日志出现顺序颠倒, 又或者日志序列提早结束。

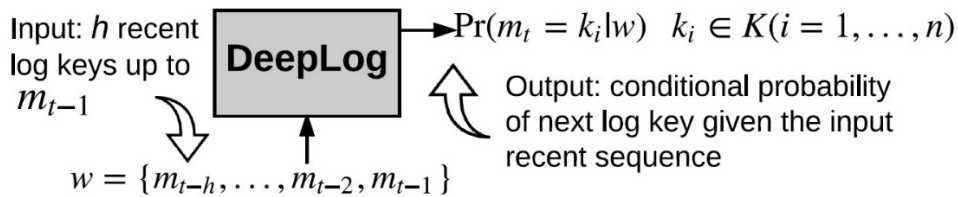


图 3: DeepLog 日志异常检测模型[17]

因此, DeepLog 使用 LSTM 去建模系统正常执行时的日志模式, 当遇到异常

的日志序列时,这个序列会因不符合 LSTM 学习到的模式而被检测到。具体来说,DeepLog 利用 LSTM 和线性层构建神经网络,该神经网络会预测输入序列后的下一条日志,利用这种方式可以学习日志的正常模式,由于模型的最终输出是一个条件概率分布,故损失函数可以选择使用交叉熵。

对于某个版本的软件系统,其源代码是固定的,那么日志事件模板的数目也是固定的。令 $K = \{k_1, k_2, \dots, k_n\}$ 为日志解析后得到的所有日志模板的集合。令 m_i 表示出现在日志序列第 i 个位置的日志模板, m_i 可能是日志模板集合 K 中的某一个。DeepLog 将日志异常检测问题建模为基于日志序列的多分类问题,如图 3 所示。假设 t 是下一条日志对应的日志序列位置,模型的输入是一个长度为 h 的窗口 w ,窗口中是距离 t 最近的 h 条日志(模板),即 $w = [m_{t-h}, \dots, m_{t-2}, m_{t-1}]$ 。窗口中的每一个 m_i 都是 K 中的元素,并且由于部分日志基于相同的日志事件模板,相同的 k_i 可能会在 w 中重复出现。窗口 w 是通过上文提到的滑动窗口(3.1.2 特征提取)得到的,其中的窗口长度就是 h 。神经网络最终的输出是一个条件概率分布 $P[m_t = k_i | w] \forall k_i \in K$ 。这个深度模型在检测阶段将会读入日志窗口,并输出预测结果,然后这个预测结果会与实际观测到的日志(模板)进行比较。

在异常检测阶段,得益于基于序列预测的检测方式,训练好的 DeepLog 模型能以在线的方式读入日志流,并且把异常定位精确到单条日志(模板)。DeepLog 维护一个日志序列,对于一条新出现的日志 e_t ,其经过日志解析后得到日志事件模板 m_t ,为了判断这条日志是否异常,DeepLog 将日志序列中的前 h 条日志事件模板取出构成窗口 $w = [m_{t-h}, \dots, m_{t-1}]$ 并输入训练好的神经网络,神经网络输出一个条件概率分布 $P[m_t | w] = \{k_1:p_1, k_2:p_2, \dots, k_n:p_n\}$,这个条件概率分布刻画了在给定长度的历史上下文下,下一条日志模板类型的可能性。

对于具体异常检测的策略,DeepLog 根据神经网络输出的条件概率 $P[m_t | w]$,对所有可能的日志事件模板 K 从大到小排序,如果 m_t 不在前 g 个候选日志模板中,则认为此条日志不符合正常模式,也就是发生了异常,反之则认为正常。 g 的选择取决于具体运维的需要, g 越小异常检测越敏感,但是也更加考验模型的精度。

2.2 基于机器学习的检测方法

基于机器学习的日志异常检测方法已经被许多学者研究过,这些方法分为有监督学习和无监督学习。有监督学习依赖于有标注的训练数据,需要严格对数据集的每个日志序列标注其为异常或者非异常。无监督学习则不依赖于有标注的数据,是一种更加通用的方法,因为实际生产环境中的数据往往都是缺少标注的。

逻辑回归是一种常用于二分类的统计学模型,其主要思想是通过一个

logistic 函数来预测某个事件发生的概率 p ($0 < p < 1$), 并将这个概率转换为一个类别标签, 通常是 0 或 1, 表示两种可能的结果。在日志异常检测中, 利用训练集中日志序列的计数向量和标签来训练逻辑回归模型, 对一个样本产生预测概率 p , 如果满足 $p \geq 0.5$ 就认为这个样本是异常的。

决策树是一种常见的有监督机器学习算法, 主要用于分类和回归任务。它维护一种树状结构, 每个内部节点表示一个属性的决策, 每个分支代表决策结果的一种可能性, 而每个叶节点表示一个类别标签或回归值。决策树使用训练数据自顶向下构建, 每个内部节点都是使用当前最具区分性的属性创建的, 该属性通常通过信息增益来选择[14]。来自加州大学伯克利分校的 Mike Chen 等人将决策树首次用于 Web 请求日志系统的故障诊断[13]。他们使用日志事件计数向量作为特征构建决策树, 叶节点表示是否异常。

支持向量机是一种常用于分类的有监督学习方法, 其核心思想是在高维空间中构建一个最优的超平面来分隔不同类别的数据点。来自罗格斯大学的 Yinglung Liang 等人将支持向量机运用于异常检测并与其他方法进行了比较[15]。使用日志事件计数向量作为数值特征, 在通过支持向量机进行异常检测时, 如果数据点位于超平面上方, 则会报告为异常, 反之视为正常。

LogCluster[3]是一种由 Qingwei Lin 等人提出的基于聚类的日志异常检测方法, 其训练需要两个阶段: 知识库初始化和在线学习。知识库初始化由三个步骤组成: 首先将日志计数向量加权求和构成日志序列向量, 接着使用层次聚类将正常和异常的样本分别聚类, 生成两组向量簇作为知识库, 最后每个簇的质心作为其代表性向量。在线学习阶段会进一步调整这些簇。在异常检测阶段, LogCluster 先计算日志向量与知识库中各代表性向量的距离, 若最短距离大于阈值, 则视该日志序列为异常。否则, 按照最近簇的异常性来报告。

主成分分析是一种常用的数据降维技术, 其核心思想是通过线性变换将原始数据映射到一个 k 维的坐标系中, k 是小于原始维度的值。新坐标系的基是原始数据的主成分。主成分按照方差递减的顺序排列, 第一个主成分捕获了高维数据中最大的方差, 第二个主成分捕获了次大的方差, 依此类推。

来自加州大学伯克利分校的 Wei Xu 等人将主成分分析运用在了日志异常检测中[4]。在日志序列被向量化为日志事件计数向量后, 使用主成分分析寻找事件计数向量维度中的模式。利用主成分分析可以生成两个子空间: 正常空间 S_n 和异常空间 S_a 。 S_n 由前 k 个主成分构成, S_a 由剩余的 $(n - k)$ 构成, 其中 n 是原始数据的维度。利用 $y_a = (I - PP^T)y$ 将日志事件计数向量投影到 S_a , 其中 $P = [v_1, v_2, \dots, v_k]$ 是前 k 个主成分。如果 $\|y_a\|^2$ 大于阈值 Q_α , 则认为对应的日志序列异常, 其中 α 代表这个阈值能提供 $(1 - \alpha)$ 的置信度。

不变量挖掘是挖掘利用那些在程序执行过程中始终保持不变的属性或关系，它们可以是程序中的常量、约束条件或者关于程序变量之间的关系。在文章[16]中，研究人员首次将不变量挖掘应用于日志异常检测。不变量挖掘能揭示多种代表系统正常行为的日志事件之间的线性关系，且这种线性关系其实是很常见的。例如，文件打开后必须关闭，则“打开文件”的日志事件和“关闭文件”的日志事件应当成对出现。

输入日志事件计数矩阵，使用奇异值分解估计不变量空间，确定下一步需要挖掘的不变量数量 r 。然后，通过蛮力搜索算法找出不变量。最后，通过将每个挖掘到的不变量候选项的支持度与阈值进行比较来验证它们。循环这个过程，直到找到 r 个独立的不变量。通过检查一个日志序列是否违反不变量，其异常与否可以被检测出来。若至少有一个不变量被违反，该日志序列将被视为异常。

2.3 基于深度学习的检测方法

利用深度学习进行日志异常检测能够提高模型的适应性，处理更大规模的数据，模型能够自动从原始数据中学习到的深刻的特征表示。因此，深度学习成为日志异常检测的重要方法之一。这些方法也分为有监督和无监督的方法，按照损失函数形式的不同又可以分为预测损失、重建损失和有监督损失。

预测损失的核心思想是，日志序列在系统正常执行的情况下遵循一定的模式。而异常发生时，这种正常的日志模式往往会被破坏。令 $K = \{k_1, k_2, \dots, k_n\}$ 为日志解析后得到的所有日志模板的集合，对于输入模型的窗口 $w = [m_{t-h}, \dots, m_{t-2}, m_{t-1}]$ ，序列预测模型最终输出一个条件概率分布 $P[m_t = k_i | w] \quad \forall k_i \in K$ 用于预测 m_t 对应的日志事件模板类型。若 m_t 实际的日志事件模板不在预测结果的前 g 名中，则认为发生了异常。

重建损失主要用于自编码器中，该模型训练的目标是让输出和输入尽量一样。具体来说，给定输入窗口 w 和模型输出窗口 w' ，重建损失可以被表示为 $sim(w, w')$ ，其中 sim 是一个相似性函数。通过让模型学习如何正确地重建正常窗口，当遇到异常日志窗口的时候，重建失效并导致损失过大。

有监督损失要求训练数据事先标注好异常与否，模型直接输出判断某个日志序列是否异常。具体来说，给定输入窗口 w 及其标签 y_w ，模型被训练以最大化条件概率分布 $P(y = y_w | w)$ 。常用的有监督损失函数包括交叉熵和均方误差。

DeepLog[17]是由Min Du等人提出的日志异常检测方法，首次将深度学习中的LSTM用于日志异常检测。Min Du等人认为，正常的日志序列遵循一定的模式。DeepLog将每条日志表示为其对应日志事件模板的编号，通过神经网络学习正常日志序列的模式，并基于预测损失来检测异常。这种方法也启发了很多后来的研

究者。

LogAnomaly[22]是由 Weibin Meng 等人提出的方法，他们在 DeepLog 的基础上进一步考虑了日志的语义。具体来说，LogAnomaly 使用称为 `template2Vec` 的方法将日志模板转化为嵌入向量：使用嵌入模型 `dLCE`[23]来生成词向量，而模板向量被计算为模板中词语词向量的加权平均值。LogAnomaly 同样使用 LSTM 模型，基于序列预测检测异常。

Loisy[24]是将 Transformer 结构应用于日志异常检测的一次尝试，这是一种基于分类的方法，通过学习日志表示来更好地区分正常日志数据和来自辅助日志数据集的异常样本。辅助数据集有助于增强正常数据的表示，并提高模型的泛化能力。异常检测的过程同样是基于序列预测。

自编码器被 Farzad 等人[25]尝试结合孤立森林用于日志异常检测。这是一种无监督模型，自编码器使用正常日志序列训练，能够提取特征，然后用于异常检测，而孤立森林则基于提取的特征进行异常检测。当遇到异常日志序列时，重建损失会相对较大，标志着发生异常。

Xu Zhang 等人[12]认为，许多现有的日志异常检测模型，没有考虑现实世界日志的不稳定性，从而导致实验中的性能无法在实际应用中兑现。因此他们提出了 LogRobust，该模型将注意力机制和双向 LSTM 结合，以便为日志事件模板分配不同的权重，称为注意力双向 LSTM。同时，为了应对日志演变和日志处理噪声，LogRobust 利用现成的词向量来提取日志事件的语义信息，对于从未见过的日志事件模板，也可以将词向量通过词频-逆文件频率加权求和作为最终的模板向量。

Siyang Lu 等人在其文章[26]中提出使用 CNN 进行日志异常检测。作者首先通过会话窗口（基于特定标志符划分）构建日志事件序列，然后通过填充或截断使各个序列长度一致。为了进行二维特征上的卷积计算，他们提出了一种称为 `logkey2vec` 的嵌入方法。具体来说，他们首先创建了一个可训练的矩阵，其形状为 $\#distinct\ log\ events \times embedding\ size$ ，其中 *embedding size* 是一个可调的超参，该矩阵能将日志事件嵌入到向量空间。然后，对嵌入后的序列（此时是一个矩阵）应用不同形状设置的卷积层，拼接其输出并输入全连接层以生成预测结果。

2.4 基于状态抽象解释 RNN

针对 RNN 的量化分析和解释方法，近年有不少相关的研究开展，而本毕业设计中的模型解释就基于 DeepStellar[18]和 DeepSeer[19]实现。DeepStellar 的核心思想是，把 RNN 建模为具有状态转换的系统。通过这种方式，DeepStellar 能在一定程度上捕捉 RNN 的内部运行细节，并对模型进行量化分析。在抽象模型

构建完毕后, 为了提供更多元的视图, 方便用户探索、理解模型, DeepSeer 提供了一套可视化的交互系统, 提供不同粒度的解释。

在相关文章 ([18][19][20][21]) 中, 许多研究者提出将 RNN 建模为具有状态转换的系统, 而这种建模的第一步就是状态抽象。RNN 是一种为了处理序列数据而生的神经网络模型, 它能够处理如文本这样的序列数据。RNN 首先需要初始化其隐向量 $h_0 \in \mathbb{R}^N$, 其中 N 是隐向量的维度, 一般可以使用全零初始化。对于一个输入的序列 $[x_1, x_2, \dots, x_T]$, 在第 t 时间步, RNN 利用输入序列的 x_t 和上一时间步的隐状态向量 h_{t-1} 来更新隐状态, 得到新的隐状态向量 h_t 。而状态抽象可以帮助人们理解 RNN 在更新“记忆”的过程中发生了什么。

像 LSTM、GRU 这样的 RNN, 在主流的深度学习框架中都是可以获取其隐藏状态的。通过将训练数据传入 RNN 进行推理, 在遍历训练数据的过程中就可以收集具体的隐向量。对于每一个输入的特征序列样本, 可以收集到具体隐向量 $[s_1, s_2, \dots, s_h]$, 其中的 h 代表序列的长度, 每一个 s_i 都是高维空间中的一个具体向量。为了降低运算的复杂性, 部分研究者选择先利用主成分分析对所有具体的隐向量进行数据降维。

经过预处理的隐向量需要再进行聚类, 常见的聚类方法有高斯混合模型、网格划分、KMeans 等。聚类的数目可以由人工决定, 也可以通过算法自动决策, 一般来说聚类的数目越多, 抽象模型越精细, 与原 RNN 的契合度也越高, 但是可解释性也会进一步下降。通过状态抽象, 模型推理过程中隐向量的更新就被体现为一个抽象状态的转换序列, 这样的状态序列可以被称为模型推理的轨迹。例如, 对于某一序列的推理过程中收集到具体隐向量 $[s_1, s_2, s_3]$ 。在收集完整个训练集的所有隐向量后进行聚类, s_1 被聚类到簇 $\hat{s}_1$, 而 s_2 和 s_3 被聚类到簇 $\hat{s}_2$, 那么这个序列的推理轨迹就是 $[\hat{s}_1, \hat{s}_2, \hat{s}_2]$, 在这里 $\hat{s}_i$ 就代表着抽象状态。状态之间的转移可以被表示为 $\delta \subseteq \hat{S} \times \hat{S}$, 其中 $\hat{S}$ 表示所有抽象状态组成的集合。

DeepStellar 借助状态抽象, 最终把 RNN 建模为离散时间马尔科夫链。离散时间马尔科夫链可以表示为一个三元组 $(\hat{S}, I, \hat{T})$, 其中 $\hat{S}$ 是所有抽象状态构成的集合, I 是一系列初始状态构成的集合, 而 $\hat{T}: \hat{S} \times \hat{S} \mapsto [0, 1]$ 是不同状态之间转换的概率。令 $P(\hat{s}, \hat{s}')$ 表示当前状态为 $\hat{s}$ 情况下下一个访问的状态为 $\hat{s}'$ 的条件概率, 这个概率可以通过简单的统计学习到:

$$P(\hat{s}, \hat{s}') = \frac{|\{(s, s') \mid s \in \hat{s} \wedge s' \in \hat{s}'\}|}{|\{(s, _) \mid s \in \hat{s}\}|}$$

DeepSeer 在此基础上, 将这个有状态系统绘制了出来, 通过多种可交互的视图让抽象状态以及模型推理轨迹更易于理解。

第三章 模型构建与系统实现

3.1 工具概览

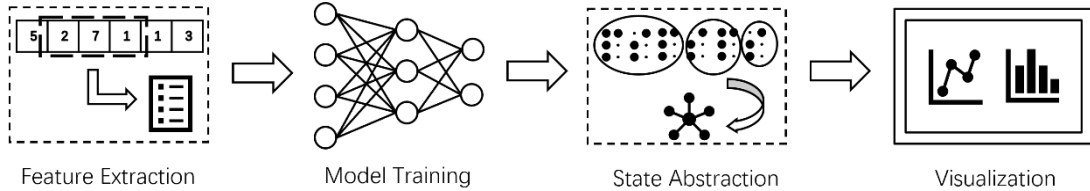


图 4: 工具流程概览

本毕业设计实现了一个日志异常检测的可解释性工具,可以帮助用户理解基于 RNN 的日志异常检测模型。这个工具包含三个部分:日志异常检测模型的训练框架,基于状态抽象的抽象模型构建工具和可交互的可视化系统。

首先,通过这个工具,用户可以训练基于 RNN 的日志异常检测模型。用户既可以复现经典的模型,也可以通过修改配置文件和扩展代码更改模型结构和日志特征,实现新的模型。训练框架能在给定的数据集上评估模型,计算模型的 *Precision*、*Recall* 和 *F-measure* 等指标,帮助用户调整超参,训练出效果更好的异常检测模型。

其次,用户可以使用状态抽象技术对训练好的模型进行简化,构建出抽象模型。抽象模型构建工具使用状态抽象技术,将复杂的 RNN 简化为只有几十个状态的离散时间马尔科夫链,在简化模型复杂度的同时,也能很大程度地提高模型的可解释性。同时,经过实验验证,简化前后的模型能在输出结果上保持较高的一致性。

最后,用户可以将构建的抽象模型展示在可视化系统上。可视化系统提供了状态图、在线推理等视图,各个视图都具有一定的可交互性,能从不同的角度帮助用户理解异常检测模型。在文章的最后将会给出一个具体的案例以说明如何对模型进行解释。

如图 4 所示,本毕业设计实现了特征提取、模型训练、状态抽象和可视化的流程。该工具能帮助用户方便地训练日志异常检测模型。用户也能使用该工具对模型做出解释,进而理解这个基于 RNN 的模型及其异常检测检测结果。

3.2 日志异常检测模型的实现

本毕业设计基于实现的训练框架,搭建并训练了 Min Du 等人提出的

DeepLog[17]模型，并基于此模型进行后续的异常检测和状态抽象，最终验证整个工具的可用性。

对于像 HDFS-v1[4] 这样的大数据集，DeepLog 在训练过程中只需要用到其中的很小一部分(1%左右)，同时因为 DeepLog 是基于日志序列预测的无监督模型，这一小部分的数据需要产生自系统无异常运行的过程。对于一个长度为 h 的日志窗口，DeepLog 的深度模型会预测下一条日志模板的类型。例如，对于一些系统正常运行产生的日志，这部分日志经过日志解析后得到的日志模板序列为： $[k_{22}, k_5, k_5, k_5, k_{11}, k_9, k_{11}, k_9, k_{11}, k_9]$ 。给定滑动窗口的大小 $h = 7$ ，那么经过滑动窗口产生的输入窗口和输出标签就是： $\{[k_{22}, k_5, k_5, k_5, k_{11}, k_9, k_{11}] \rightarrow k_9\}$ ， $\{[k_5, k_5, k_5, k_{11}, k_9, k_{11}, k_9] \rightarrow k_{11}\}$ 和 $\{[k_5, k_5, k_{11}, k_9, k_{11}, k_9, k_{11}] \rightarrow k_9\}$ 。

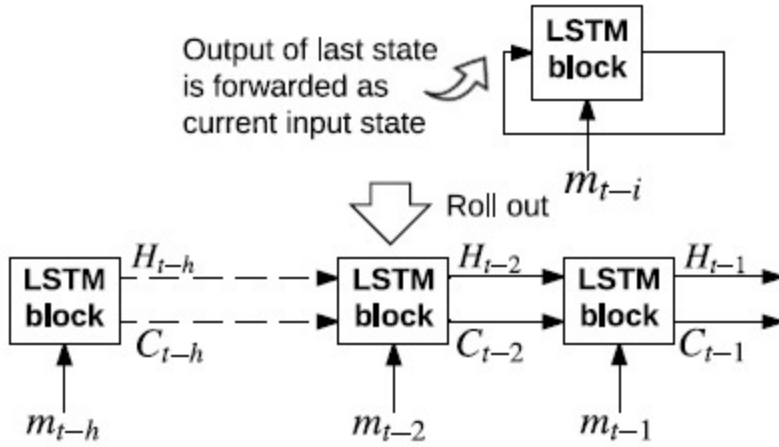


图 5: LSTM 循环视图[17]

RNN 是自然语言处理中常用的神经网络，而基于 LSTM 的模型可以学习到更复杂模式，适用于处理具有长期依赖关系的序列数据。LSTM 具有特殊的结构，包括输入门、遗忘门、输出门和记忆单元。这些门控制了信息在网络中的流动和存储，使得 LSTM 能够有效地学习和记忆长期依赖关系。DeepLog 的原论文中涉及到使用多个 LSTM 模块堆叠构建神经网络，而在本毕业设计中，为了方便后续的基于状态抽象的模型解释，只使用了单层的 LSTM。

图 5 显示了单个 LSTM 模块的循环过程：每个 LSTM 模块会以隐向量的形式记忆状态，来自上一个时间步的 LSTM 模块的隐向量也会作为下一步输入的一部分，与外部数据输入一起计算出一个新的隐向量和输出。历史上下文信息以隐向量的形式被保持，并在一个个时间步中不断传递变化。将 LSTM 模块循环的过程在时间步上展开，每个时间步上的 LSTM 模块维护一个隐向量 H_{t-i} 和一个细胞状态向量 C_{t-i} ，这两个向量被传递给下一个时间步的 LSTM 模块，并和下一时间步的输入 m_{t-i+1} 一同计算新的输出和隐状态。在单个 LSTM 模块内部， m_{t-i+1} 和 H_{t-i} 被

用于决定之前的细胞状态 C_{t-i} 有多少被保留到 C_{t-i+1} ，以及新的输出 H_{t-i+1} 的生成。这一切是通过一组门控函数来实现的：通过控制要从外部输入和上一个输出中保留的信息量以及传递到下一个时间步的信息流，LSTM 能确定状态如何变化。每个门控函数都有一组需要学习的权重参数，LSTM 捕捉细节的能力取决于隐向量 H 的维度， H 的维度越大，能捕捉到的信息越多。

在具体的代码实现中，我选择使用目前流行的深度学习框架 PyTorch 来实现 DeepLog 中的神经网络。借助 PyTorch 中的 LSTM、Linear 类和 softmax 函数，DeepLog 的神经网络主体能在几十行 Python 代码内实现。虽然实验中主要使用 Min Du 等人提出的 DeepLog 模型，但是代码实现中我仍然考虑了可扩展性。具体来说，我实现了一个特征提取器 FeatureExtractor，承担窗口划分、特征提取等任务，可以通过简单修改配置以提取不同类型的特征。

3.3 基于状态抽象的抽象模型构建

为了收集 RNN 的隐向量，基于 RNN 的模型需要有一个函数，用于对外暴露 RNN 模块在推理过程中产生的隐向量。在本毕业设计中，我在实现 DeepLog 的同时为模型添加了 profile 函数，将 torch.nn.LSTM 产生的隐向量暴露出来，profile 函数输入日志窗口，将同时返回推理中每一步的隐向量和最终模型的分类型预测结果。

将预训练好的 DeepLog 模型以及训练时使用的特征提取器加载后，利用 profile 函数迭代整个训练集，过程中收集隐向量以及对应的分类预测结果。利用收集到的隐向量，接下来可以进行状态抽象。本毕业设计基于 DeepStellar [18] 的方法进行状态抽象，因为这种方法经过研究者验证，在神经网络的调试和修复中都具有一定的效果，并且构建的抽象模型在测试集上也能表现得和原 RNN 十分接近。为了降低后续运算的复杂度，还需要对收集到的具体隐向量使用主成分分析进行数据降维。接着，对降维后的隐向量进行聚类，聚类的每一个簇代表一个抽象状态。在相关的研究中 ([18][19][20][21])，不同的聚类方法被尝试应用于状态抽象过，包括网格划分、高斯混合模型、KMeans 等。基于网格划分的聚类难以处理落在网格空间外的数据点，同时，若对 d 维的向量的每个维度划分 m 个区间，将会产生 m^d 个格子，如此庞大数量的抽象状态不利于后续的可视化。而高斯混合模型虽然具有很多优点，但是其算法复杂，在数据量较大的情况下运算十分缓慢，虽然有相关论文提出小批次迭代的高斯混合模型，但是实现复杂，总体而言不利于后续实验开展。因此，最终我选择使用 KMeans 聚类方法。

KMeans 模型训练完成后，在用于聚类时相当于一个从隐向量空间到抽象状态空间的映射，即 $C: \mathbb{R}^d \mapsto \hat{S}$ 其中 d 是降维后的隐向量的维度。 d 和 $|\hat{S}|$ 是两个可

以调整的超参，一般来说 d 越大，保留原始隐向量的信息越多，越有利于模型的训练。而 $|\hat{S}|$ 越大，表示聚类产生的簇（抽象状态）越多，抽象模型越精细，与原 RNN 的契合度也越高，但是其可解释性也会进一步下降。并且，这两个超参的变大也会消耗更多的运算时间，因此需要充分权衡效果与成本，并设置合理的参数。具体实现中这两个参数是从配置文件读入的，用户可以通过简单修改配置文件来调整，达到想要的效果。

获取抽象状态后，模型推理过程中隐向量的更新可以被表现为抽象状态的转换序列，这样的状态序列可以被称为模型推理的轨迹。接着统计出状态转换矩阵 T ，具体来说 $T_{\hat{s}_i, \hat{s}_j}$ 代表在整个训练集中，出现从状态 $\hat{s}_i$ 转换到状态 $\hat{s}_j$ 的频率，对矩阵的每一行归一化后，可以将 $T_{\hat{s}_i, \hat{s}_j}$ 视为 $P(\hat{s}_i, \hat{s}_j)$ 。最后，利用流行的图网络库 NetworkX 生成有向图的数据结构，为了在图中体现状态转换的频率，我将边的权重与其状态转换频率相关联，状态转换的频率越高，其对应边的权重越大。

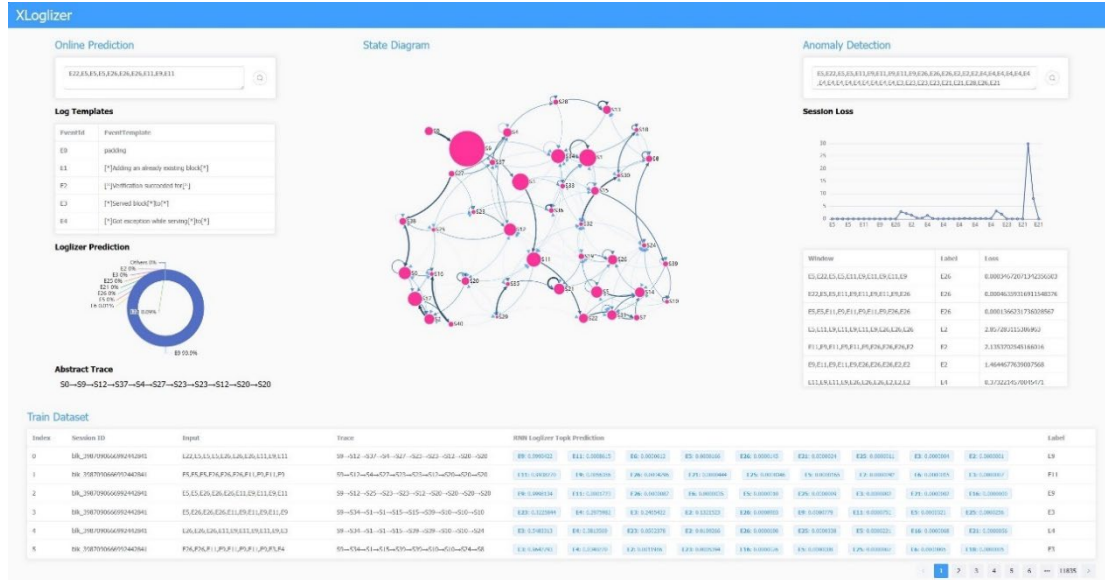


图 6: dashboard 概览

3.4 可视化系统概览

基于状态抽象构建出抽象模型后，为了让用户更好地理解 RNN 模型，我学习了 DeepSeer 的方法，将构建的图数据结构以状态图的形式渲染出来。状态图中每一个节点代表由一系列近似的隐向量聚类而成的簇，状态之间的转换通过弧形有向边体现，如图 6 所示。相比于原 RNN 动辄成百上千的参数数量，这里由数十个节点组成的状态图更容易让人理解。除此之外，在 dashboard 中，我还添加了在线的模型推理和日志序列异常检测等视图，帮助用户在交互中更好地理解模型。

3.4.1 抽象状态图与训练集视图

抽象状态图将预先构建的有限状态系统可视化, 这个有限状态系统则是由给定的基于 RNN 的模型抽象出来的。状态图可以让用户从全局视角观察模型, 同时

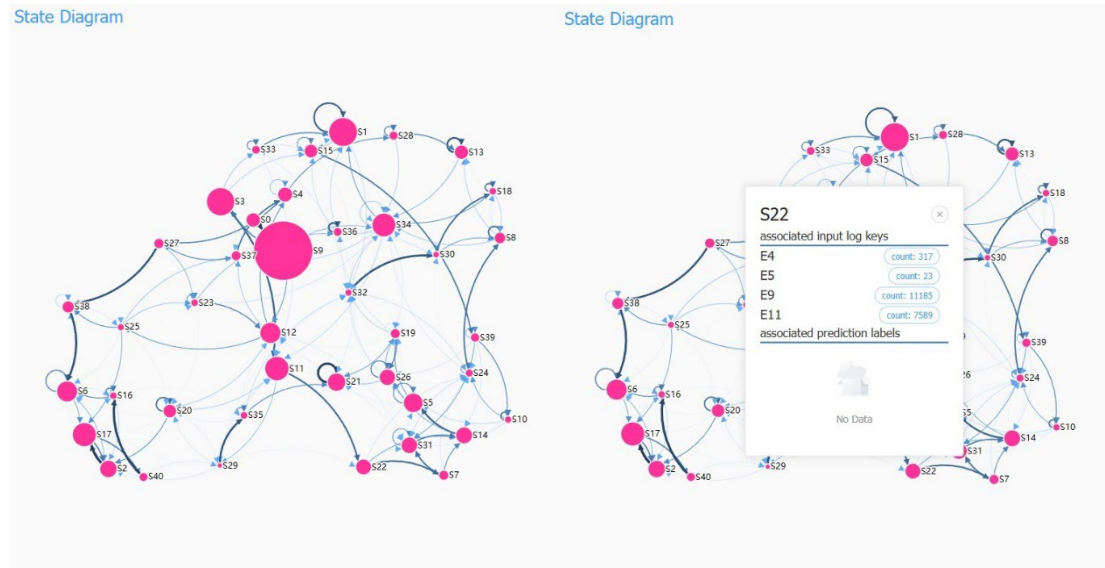


图 7: 状态图以及节点信息面板

也能帮助用户理解每一个抽象状态背后隐藏的语义。如图 7 所示, 用户可以拖拽状态图的节点, 点击状态图的节点会打开一个节点信息面板, 面板上显示该节点对应的抽象状态, 用户还可以看到输入什么日志会导致访问这个节点, 以及这个节点通常输出预测什么类型的日志事件模板。

节点的大小与训练过程中这个节点被访问了多少次相关, 输入日志窗口后模型推理的轨迹中出现节点对应的抽象状态, 就视为访问了该节点。例如, 从图 7 的左半部分可以看到, 节点 S9 的大小显著大于节点 S3 的大小, 而节点 S3 的大小大于初始节点 S0。节点 S0 的访问次数取所有节点被访问次数的平均值, 反应一种平均的访问概率。这说明在训练过程中, 大部分样本的推理轨迹都会经过节点 S9, 而节点 S3 有较大概率被访问, 可以理解为 S9 是类似初始化的节点, 而 S3 是匹配到正常日志的常见模式。

边的宽度和颜色的深浅反应这条边对应的状态转换的重要程度, 这种状态转换在出发节点的所有状态转换中的占比越大, 其对应的有向边颜色就会越深, 宽度也会越宽。例如在节点 S32 的出边中, 从 S32 到 S30 的边是最显眼的, 这代表着如果匹配到 S32 记忆的模式, 接下来窗口的下一个条日志和已检查过的日志很容易构成 S30 记忆的模式, 暗示着一种训练数据中特定的规律。

如图 8 所示, 训练集视图是一个可以翻页的表格, 用于展示训练集的输入、

标签、模型预测结果等。表格的每一行是单个数据实例，各个列包括：索引、日志序列的标志 ID、日志窗口、状态轨迹、预测结果和实际标签。用户既可以通过翻阅实例了解训练数据的构成，了解正常日志的常见模式，理解模型通过训练数据学习到了什么。同时，每一个实例还提供了模型的状态轨迹，结合状态图，用户可以了解到模型推理通常的轨迹，理解模型的重要状态和轨迹。

Train Dataset														
Index	Session ID	Input	Trace	BNN Logit2 Logit Prediction										Label
0	10_7007000699242091	022,055,02,03,04,05,06,07,08,09,10,11,12	00-02-007-04-027-002-003-002-000-020	0.9790012	0.911300012	0.613000012	0.210000012	0.261000012	0.210000012	0.261000012	0.210000012	0.261000012	0.210000012	09
1	10_7007000699242091	022,055,02,03,04,05,06,07,08,09,10,11,12	00-02-007-04-027-002-003-002-000-020	0.9790012	0.911300012	0.613000012	0.210000012	0.261000012	0.210000012	0.261000012	0.210000012	0.261000012	0.210000012	09
2	10_7007000699242091	022,055,02,03,04,05,06,07,08,09,10,11,12	00-02-007-04-027-002-003-002-000-020	0.9790012	0.911300012	0.613000012	0.210000012	0.261000012	0.210000012	0.261000012	0.210000012	0.261000012	0.210000012	09
3	10_7007000699242091	022,055,02,03,04,05,06,07,08,09,10,11,12	00-02-007-04-027-002-003-002-000-020	0.9790012	0.911300012	0.613000012	0.210000012	0.261000012	0.210000012	0.261000012	0.210000012	0.261000012	0.210000012	09
4	10_7007000699242091	022,055,02,03,04,05,06,07,08,09,10,11,12	00-02-007-04-027-002-003-002-000-020	0.9790012	0.911300012	0.613000012	0.210000012	0.261000012	0.210000012	0.261000012	0.210000012	0.261000012	0.210000012	09
5	10_7007000699242091	022,055,02,03,04,05,06,07,08,09,10,11,12	00-02-007-04-027-002-003-002-000-020	0.9790012	0.911300012	0.613000012	0.210000012	0.261000012	0.210000012	0.261000012	0.210000012	0.261000012	0.210000012	09

图 8: 训练集视图

3.4.2 在线预测与异常检测结果

如果说抽象状态图能帮助用户从整体上理解模型，那么 dashboard 提供的 Online Prediction 和 Anomaly Detection 视图能更好地帮助用户从更小的粒度去理解模型的判断，如图 9 所示。

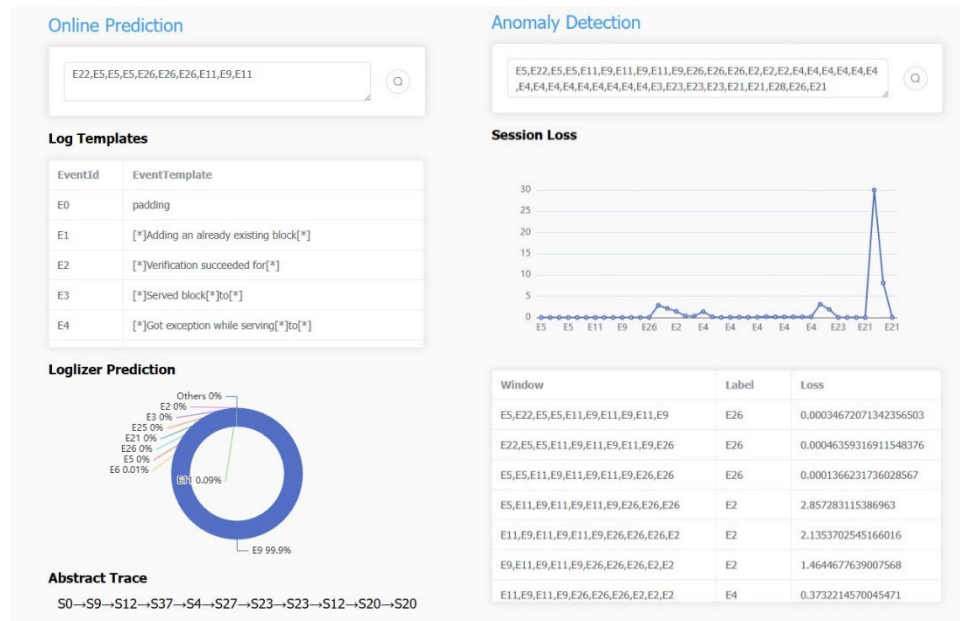


图 9: 在线推理以及异常检测

为了能让用户理解单个样本的推理过程，在线预测视图中提供了输入框，允许用户按照一定格式输入自己想要的日志事件模板序列。输入框下方的表格列出了所有的日志事件模板及其编号，用户只需要用逗号分隔编号并输入框内，点击搜索按钮，日志事件模板表格下方的环形图就会展示预测结果的前几名，同时最

下方也会显示模型推理的轨迹。RNN 通常在读取整个序列后使用最终的隐向量来计算条件概率，而用户可以利用推理的轨迹，结合状态图查看模型的推理过程中的隐状态是如何一步步变化的。

针对日志异常检测这一场景，异常检测视图允许用户从整个日志序列的角度理解异常检测的判断。利用 DeepLog 基于序列预测的特点，异常检测定位的粒度可以达到单个日志级别。之前提到过，如果实际的日志事件不在模型预测结果的前 g 名候选中，则视为发生了异常。而这个 g 是由用户自己决定的阈值， g 越小模型越敏感，但是对模型精度和正常日志模式都有一定要求。不同的用户对 g 的需求不同，因此 dashboard 上使用预测结果的交叉熵损失来指示异常，过高的损失代表预测结果与实际相差越大，通过将各个滑动窗口的预测情况展现为折线图，用户能更直观地看出异常的峰值，定位到具体的日志。折线图下方的表格罗列出各个日志窗口的预测情况，用户可以复制窗口的序列到在线推理视图中运行，查看模型的判断结果的推理过程。

第四章 实验验证与效果分析

4.1 日志异常检测效果

在本毕业设计中，我使用 PyTorch 实现了 DeepLog 模型，在这一节中，我将对实现的模型开展实验验证，评估 DeepLog 各个方面的性能，验证其是否可以在大型日志数据集上取得良好的效果。

4.1.1 实验环境设置

本次实验的所有程序都运行在一台 Windows10 操作系统的主机上，CPU 型号为 i7-13700KF，GPU 型号为 RTX4080，内存共 32GB。实验基于 conda 管理 Python 虚拟环境，使用到了 Python 3.10.14 版本，主要依赖的库包括 PyTorch、NumPy、NetworkX 等。同时实验的前端环境通过 nvm-windows 进行管理，使用到了 Node.js 20.11.1 版本，主要依赖的库包括 Vue.js、ElementPlus、Axios 等。

在数据集方面，HDFS-v1 数据集被许多研究者所使用，已经成为了日志异常检测研究中的代表性数据集。该数据集是通过在 200 多个 Amazon EC2 节点上运行基于 Hadoop 的 MapReduce 任务生成的，并经过了 Hadoop 领域专家标注。在收集的 11,197,954 条日志条目中，大约 2.9% 是异常的，包括“写入异常”等事件。HDFS-v1 数据集经过一定的预处理，有利于实验的开展。具体来说，日志条目被根据 HDFS 日志的标识字段 block ID 分组到不同的会话窗口中，每个会话窗口分别代表一个块的生命周期。同时，每一个日志条目被解析后，可以用日志事件模板的编号来代替，最终一个块的声明周期可以用一个日志模板编号序列来表示。

依照原论文，DeepLog 只需要一小部分正常日志数据用于训练。在具体实验中，我只使用了 7475 个正常会话作为训练集，1150 个正常会话作为验证集，剩下的 549598 个会话（其中有 16838 个异常会话）则全部作为测试集。训练集和验证集只占总体数据的 1.5%。

Min Du 等人认为，虽然 DeepLog 可以精确定位哪条日志（及其对应的日志事件模板）是异常的，但为了使用相同的度量标准与其他方法进行比较，而使用了“会话”作为异常检测的粒度。也就是说，只要会话 C 中至少有一个日志事件模板被检测为异常，则会话 C 被认为是异常会话，最终统计的时候也是以会话作为最基本的检测单位[17]。

日志异常检测中，通常需要考虑的指标包括：假阳性数目（ FP ）、假阴性数

目 (FN)、精度 ($Precision$)、召回率 ($Recall$) 和 F 值 ($F-measure$)。这里的精度被定义为:

$$Precision = \frac{TP}{TP + FP}$$

其中 TP 代表的是真阳性数目, 精度衡量在所有检测到的异常中真正异常的比例。而召回率被定义为:

$$Recall = \frac{TP}{TP + FN}$$

召回率衡量数据集中成功被模型检测出来的异常的比例。F 值的定义为:

$$F-measure = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

F 值是精度和召回率的调和平均数。

在模型的参数方面, 我选择了和原论文接近的 $g=9$, $h=10$, $L=1$ 以及 $\alpha=64$, 并尝试复现模型在论文中的效果。在这里, g 代表输出预测中被认为是正常日志的阈值, 即如果实际日志模板类型不在前 g 名候选者中, 则认为是异常。 h 代表了日志窗口的长度, L 代表 LSTM 模块堆叠层数, α 代表 LSTM 模块中记忆单元的个数。

4.1.2 测试结果展示

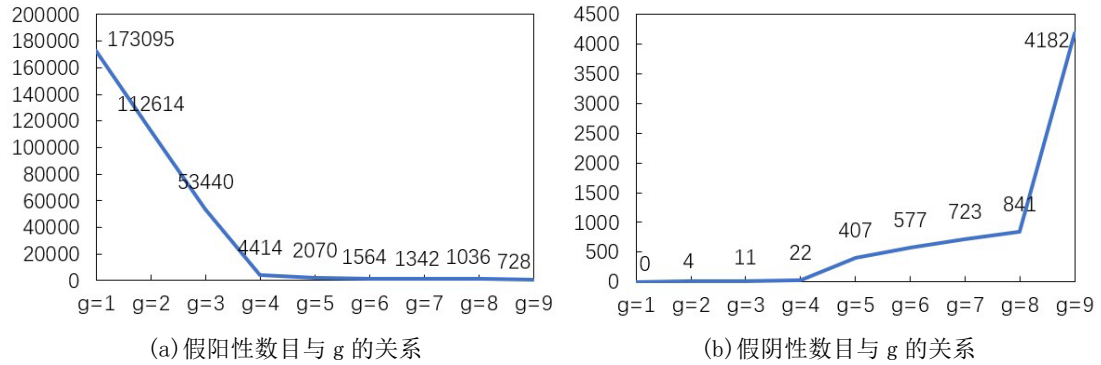


图 10: 模型在 HDFS-v1 数据集上的假阳性和假阴性表现

图 10 展示了 DeepLog 模型在 HDFS-v1 数据集上的假阳性和假阴性情况, 图 10(a)展示了假阳性数目和 g 的关系, 图 10(b)展示了假阴性数目和 g 的关系。

可以看到, 随着 g 的增大, 假阳性的数目不断减少, 这是因为 g 的增大让模型的异常检测不那么敏感, 概率分布中可能的候选者越多, 实际的日志事件模板种类更有可能落在前 g 名内, 也就不会被报告为异常。例如, 有某一种日志模式,

在训练样本中有 50% 的情况下紧接着日志事件模板 E3, 50% 的情况下紧接着日志事件模板 E4, 这意味着通过这个训练集训练出来的模型, 面对这种模式的日志窗口, 可能会输出 $P(m_t | w) = \{k_3:0.5, k_4:0.5\}$ 。如果只取 $g=1$, 则相当于误报了另外 50% 的也是正确的情况, 就导致大量的假阳性。

同样, 随着 g 的增大, 模型越来越宽容, 很可能导致部分异常的样本被视为正常。例如, 对于某一种日志模式, 模型在训练后认为, 有 32% 的情况会紧接着日志事件模板 E23, 有 30% 的情况紧接着 E4, 有 24% 的情况紧接着 E35, 有 13% 的情况紧接着 E2, 剩下的情况都是发生了异常。如果取 $g \geq 5$, 则会把一部分异常的样本也视为正常。在 $g=8$ 的情况下, 假阳性的数目和假阴性的数目分别为 1036 和 841, 与论文[17]中的效果只有比较小的差距。

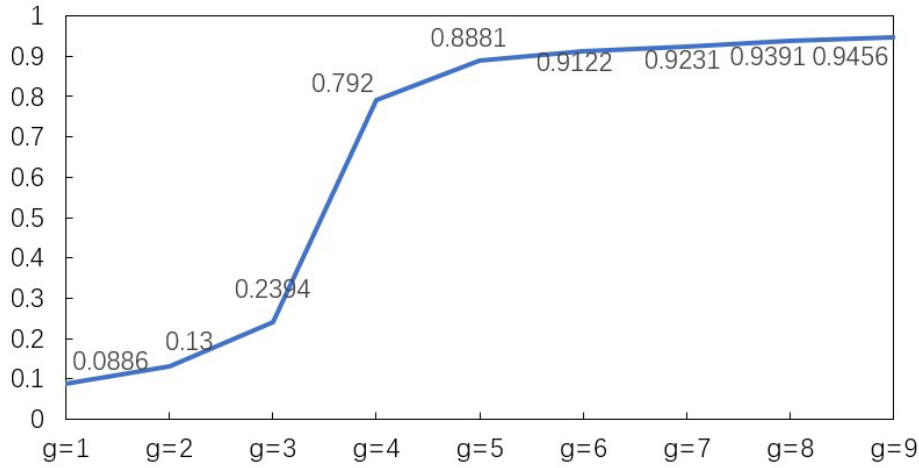


图 11: 模型在 HDFS-v1 数据集上的 Precision 表现

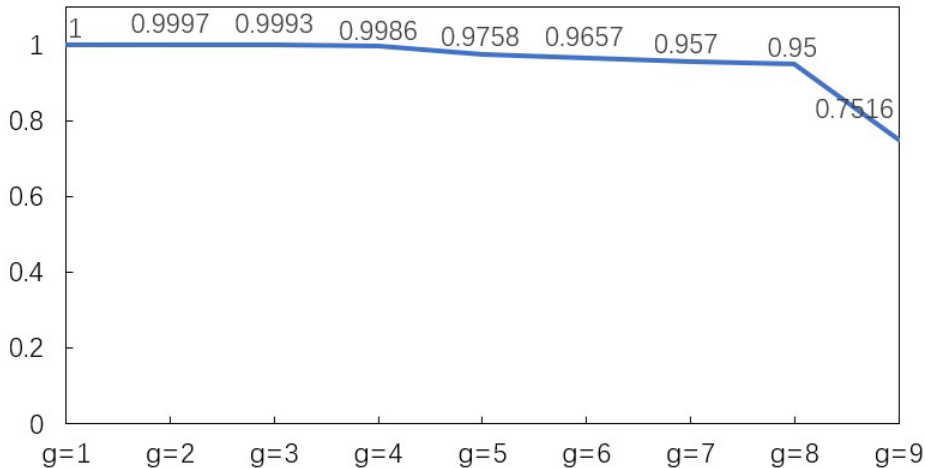


图 12: 模型在 HDFS-v1 数据集上的 Recall 表现

图 11 展示了 DeepLog 在 HDFS-v1 数据集上的精度与 g 之间的关系。通过折线图可以观察到, g 在较小的时候异常检测的精度很低, 说明此时模型的敏感度过高, 容易误报异常。从 $g=3$ 到 $g=4$ 异常检测的精度有了很大的提升, 而从

$g=5$ 开始, 模型的精度提升就较小, 最终在 $g=9$ 的时候取得最大值。

图 12 展示了 DeepLog 在 HDFS-v1 数据集上的召回率与 g 之间的关系。通过折线图可以观察到, g 从小到大变化, 异常检测的召回率始终在较高的水平, 直到 $g=9$ 的时候, 召回率才下降到 75.16%。结合假阴性与 g 之间的关系, 两张折线图都是在 $g=8$ 的地方发生了比较大的转变, 说明当 g 超过 8 的时候, 会让异常检测的容忍度过高, 最终导致许多异常样本没有被检测出来。

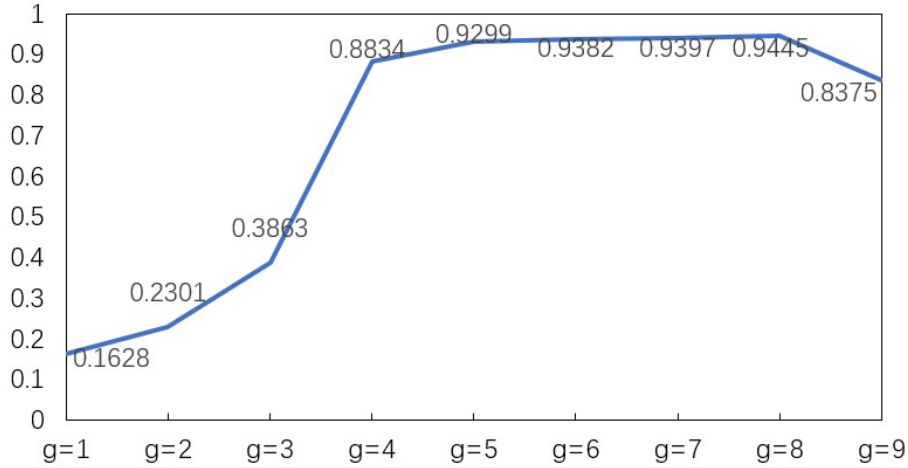


图 13: 模型在 HDFS-v1 数据集上的 F-measure 表现

图 13 展示了 DeepLog 在 HDFS-v1 数据集上的 F -measure 与 g 之间的关系。 F -measure 是精度和召回率的调和平均数, 能综合地衡量模型的性能。通过观察折线图, 不难发现当 g 过小或过大时, 都不能取得整体上最好的效果, 模型异常检测的容忍度需要被控制在一定的范围内, 折线图上 $g=8$ 的时候, 取得了最好的 94.45%。在原论文中, DeepLog 取得了精度 95%, 召回率 96% 以及 F -measure 96% 的好成绩, 本次实验使用了相似的模型参数, 并且结果与原论文差距很小, 基本验证了 DeepLog 模型在大型日志数据集上异常检测的有效性。

4.2 抽象模型效果

状态抽象的过程包括两步: 首先, 对收集到的一系列隐向量使用主成分分析进行降维; 然后, 使用 KMeans 方法聚类得到簇, 一个簇构成一个抽象状态。这个过程涉及到两个参数: 主成分分析降维后向量的维度 d 以及聚类簇的数目 n 。在本次实验中, 这两个参数被设置为 $d=32$ 以及 $n=40$ 。一般来说, d 越大, 原始隐向量能被保留下来的信息就越多, 但是聚类 and 后续处理所需的计算时间也越久。 n 越多, 抽象模型的状态越多, 抽象模型越精细, 也越能详细展现原 RNN 模型的内部行为, 但是可解释性会进一步下降。

一个被许多研究者使用过的指标 *fidelity*，能够衡量抽象模型与原 RNN 模型能否做出一致的决策，能在多大程度上做出一致的决定。假设有数据集 $X = \{x_1, x_2, \dots, x_N\}$ ，抽象模型被表示为 $\mathcal{A}$ ，而 RNN 模型被表示为 $\mathcal{R}$ ，那么 *fidelity* 就被定义为：

$$fidelity = \frac{\sum_{i=1}^N \mathbb{1}(\mathcal{A}(x_i) = \mathcal{R}(x_i))}{N}$$

在这里， $\mathcal{A}(x_i)$ 的含义是，对 x_i 使用训练好的主成分分析模型降维，并对序列的各个向量使用训练好的 KMeans 聚类模型聚类，选取序列最后一个抽象状态，利用构建的抽象模型的统计信息，将这个状态对应的最常出现的预测结果作为 $\mathcal{A}(x_i)$ 的结果。

最终，对于 DeepLog 这样的序列预测多分类模型，仍然能达到 *fidelity* 87.83% 的效果，该结果能说明抽象模型与原 RNN 模型的预测结果有一定的一致性。在实验中，经过多次尝试， $40 < n < 100$ 的时候，*fidelity* 并没有明显的提升，因此选择 $n = 40$ 能平衡抽象模型的复杂性和精度。

4.3 可视化系统使用场景

本节将针对可视化系统的效果详细展开，并结合案例说明其使用场景。dashboard 提供了状态图、在线预测、异常检测和训练集视图，通过将这几个部分合理结合使用，可以让用户更好地理解模型以及异常检测结果。

在 HDFS-v1 中，选择 block ID 为 blk_-8531310335568756456 的块，将所有与这个块相关的日志按时间顺序组成会话窗口，这个会话窗口也就反映了这个块的整个声明周期。将这个会话序列输入 dashboard 的异常检测视图的输入框内，点击搜索按钮，后端会自动对该序列执行滑动窗口，并将各个窗口逐个输入模型进行推理，得到预测结果与预测损失。最后，输入框下方将利用各个窗口的损失值渲染出一个折线图。如图 14 所示，折线图中出现了一个峰值，这代表该点的

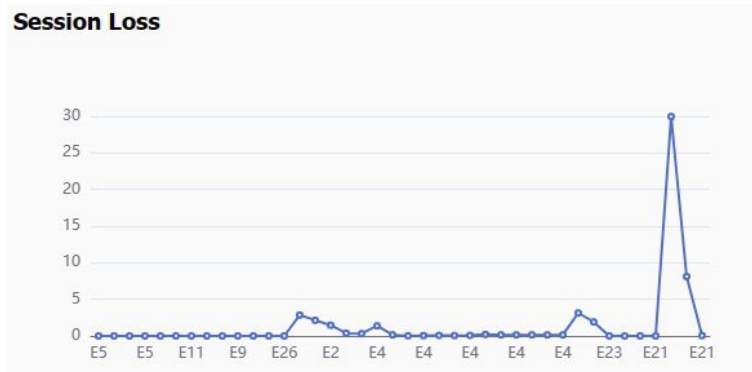


图 14: blk_-8531310335568756456 会话序列的预测损失折线图

预测结果脱离实际情况，很可能发生了异常。

Window	Label	Loss
E4,E4,E4,E4,E4,E4,E4,E3,E23	E23	0.001609009806998074
E4,E4,E4,E4,E4,E4,E4,E3,E23,E23	E23	0.0001667877077125013
E4,E4,E4,E4,E4,E4,E3,E23,E23,E23	E21	0.0005510775372385979
E4,E4,E4,E4,E4,E3,E23,E23,E23,E21	E21	0.00012279310612939298
E4,E4,E4,E4,E3,E23,E23,E23,E21,E21	E28	29.984941482543945
E4,E4,E4,E3,E23,E23,E23,E21,E21,E28	E26	8.11612319946289
E4,E4,E3,E23,E23,E23,E21,E21,E28,E26	E21	0.046600524336099625

图 15: blk_-8531310335568756456 会话序列各窗口与预测损失值

在折线图下方的表格中，可以看到具体的日志窗口和损失情况。如图 15 所示，窗口 [E4, E4, E4, E4, E3, E23, E23, E23, E21, E21] 后实际的下一个日志事件模板是 E28，而预测损失达到了 29.9849。利用在线预测的功能，可以进一步观察其具体的预测结果。点击这一行的“Window”列，将这个日志窗口复制到剪贴板中。将日志窗口粘贴到在线预测的输入框中，点击搜索按钮，后端会将这个窗口输入模型进行推理并返回预测结果。

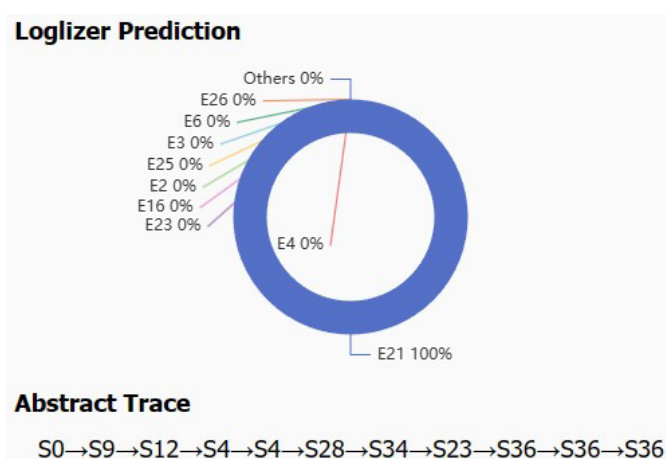


图 16: 日志窗口预测结果

如图 16 所示，在线预测日志事件模板表格的下方会渲染一个环形图，环形图上标出了概率最高的几种可能性。可以看到，输入这个序列后，模型几乎 100% 认为下一个日志事件模板应当为 E21，而且前几名候选结果中也并没有 E28。此时只要用户预期的 g 是大于 9 的，就可以认为发生了异常。根据 4.1.2 中的内容，DeepLog 在 HDFS-v1 数据集上取 $g = 9$ 是比较合理的，因此在这里可以认为系统发生了异常。

环形图的下方列出了模型推理的状态轨迹,用户可以沿着这个轨迹在状态图上查看相关节点。例如,模型读入 E4 转换到 S9,相当于 RNN 模型进入判断状态,再读入 E4 转换到 S12,观察 S12 的节点信息,发现与之相关的输入主要是 E5、E11、E4 和 E3。通过翻看训练集视图,可以发现这几种日志事件模板都是经常出现在日志窗口第二个位置上的,说明 S12 学习到的主要是那些 E5、E11、E4 和 E3 出现在日志窗口第二个位置上的模式。

接着读入后续两个 E4,可以发现状态转换到 S4 并在 S4 上自环了一次,通过观察 S4 的节点信息,可以推测 S4 也学习到了多个 E3、E4 反复出现的这种模式。接着读入 E3,状态转换到 S28,观察 S28 的节点信息,发现与之相关的输入主要是 E3 和 E4,结合浏览训练集视图,观察到 E3 或 E4 连续出现是十分常见的模式,可以进一步推测 S28 记忆了 E3 或 E4 交替出现 5 次及以上的这种模式。为了验证这个想法,在该窗口 E3、E4 反复出现中的那段中,随机加入一个 E3 并删去末尾的 E21(保持窗口长度为 10),得到新的窗口,使用该窗口进行在线预测。

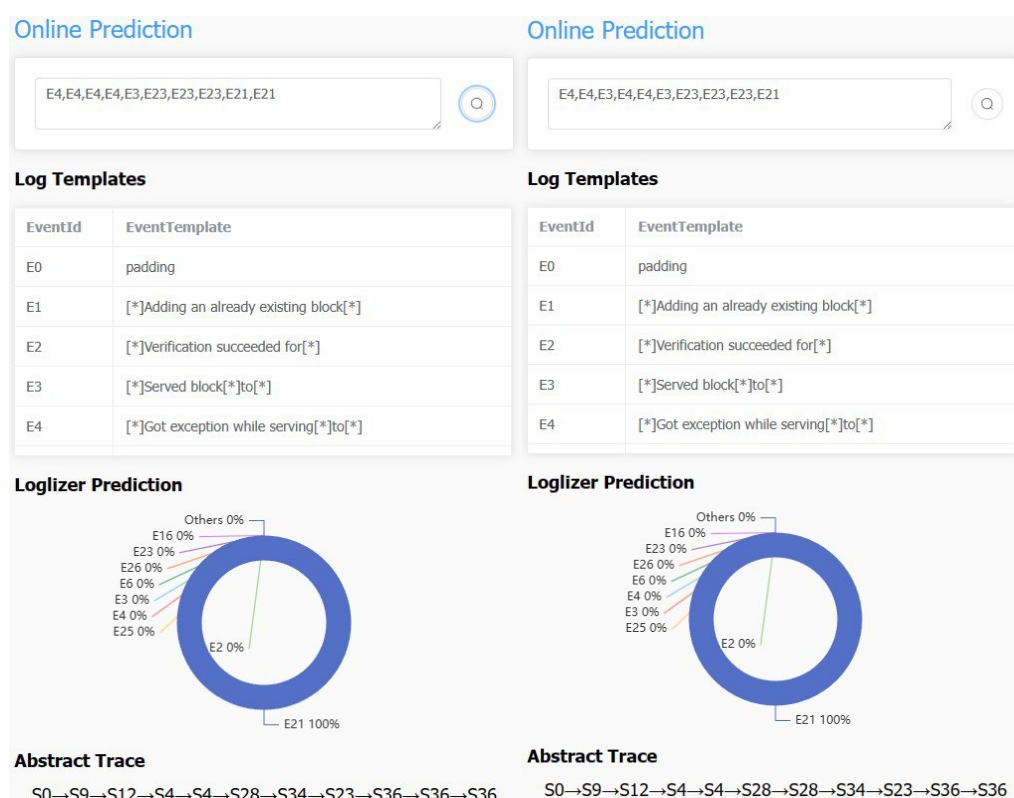


图 17: E3、E4 反复出现的模式及其状态轨迹

如图 17 所示,状态轨迹的前面一部分仍保持为“S0→S9→S12→S4→S4→S28”,由此可以说明 S28 确实是 RNN 学习到多个 E3、E4 交替出现一定次数的模式的一种表现。而且观察修改过的窗口[E4, E4, E3, E4, E4, E3, E23, E23, E23, E21],“E4, E4, E3, E4, E4, E3”部分对应“S0→S9→S12→S4→S4→S28→S28”部分的状态轨迹,更进一步说明了 S28 记忆的是 E3 或 E4 交替反复出现 5、6 次的这种模

式。

回到原本的[E4, E4, E4, E4, E3, E23, E23, E23, E21, E21]日志窗口, 读入模型读入第 6 个日志事件模板 E23, 状态转换到 S34。如图 18 所示, 观察 S34 的节点信息, 发现与之主要相关的输入就是 E23。不难推测 S34 学习到的是那些与出现 E23 息息相关的模式。

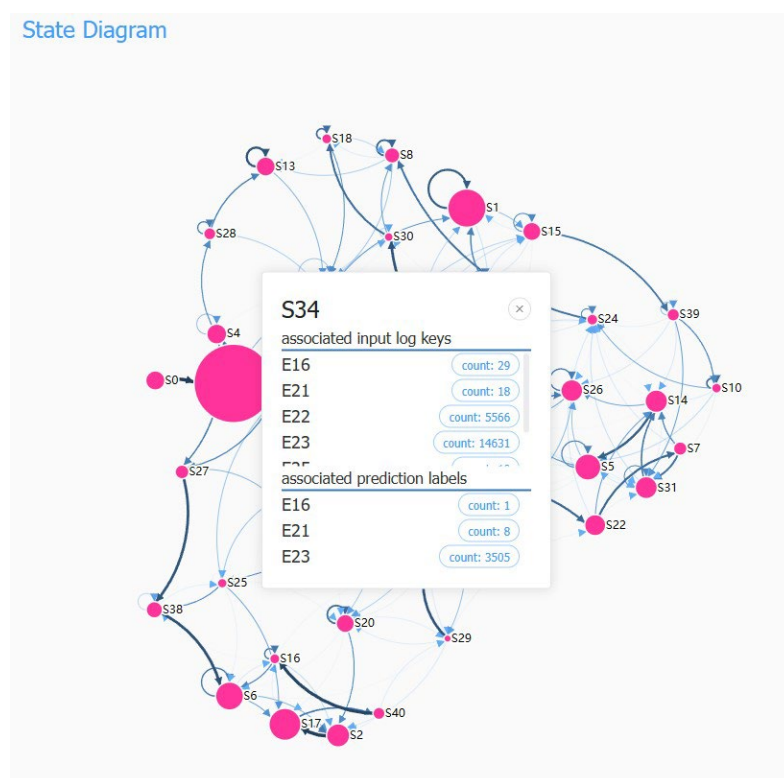


图 18: S34 节点信息面板

之后模型连续读入两个 E23, 对应状态轨迹的“S34→S23→S36”, 可以推测 S23 和 S36 两个状态也是与 E23 相关。此时观察训练集视图, 不难发现连续的 E23 跟着连续的 E21, 是一种常见的模式, 而且总是伴随着一个块生命周期的结束。结合所观察的日志窗口的最后两个日志事件模板均为 E21, 读入后对应状态轨迹的“S36→S36→S36”, 可以认为 S36 学习到的就是这种连续 E23 紧接连续 E21 的模式。结合 E23 的语义“块被标记为无效, 即将从文件系统中删除”, E21 的语义“删除了与某文件相关的块”, 可以认为这种模式就是销毁块资源的过程。

至此, 就能结合整个日志窗口的语义与来解释异常的发生。日志窗口的前面一段主要是 E3 和 E4 的往复出现的模式, E3 代表块被读取、传输提供了服务, E4 代表块服务过程中发生了意外。发生意外的原因可能是传输失败、写入失败等, 这些往往是网络和磁盘资源的不足导致的。一个块可能会被多个节点所需要, 而且节点也可能会多次尝试请求块, 因此在这个块提供服务的生命周期中, E3 或 E4 连续出现是正常现象。

E23 的出现意味着在 HDFS 的 NameNode 中，一个数据块被标记为无效，并被添加到某个无效集中。这通常意味着这个数据块需要被删除或重新处理，也就是结束其生命周期。而 E21 的出现表示数据块正在被删除。因此连续的 E23 代表这个块被 NameNode 标记为无效，而连续的 E21 表示这个块相关的文件资源开始被释放，而且资源的释放总是标记数据块无效这个操作彻底结束后开始。

理论上，日志窗口 [E4, E4, E4, E4, E3, E23, E23, E23, E21, E21] 的后续应该只有 E21 出现，但实际上却紧接着 E28，E28 在日志模式上和其本身语义上都在昭示着异常。首先，E28 的出现不符合模型学到的模式。其次，E28 的语义是“系统接收到了一次请求，试图将一个数据块添加到文件系统中，但发现该数据块并不关联到任何已知的文件”。考虑到这个数据块正处在被删除的过程中，所以系统不应该收到与之相关的 addStoredBlock request。

在 dashboard 上交互的过程，能帮助用户快速理解异常检测的结果，同时探索基于 RNN 的日志异常检测模型学习到的日志模式。将日志的模式和语义相结合，能帮助用户在模型整体和模型细节上都取得更好的理解，帮助运维人员提升运维效率。

第五章 总结与展望

本文章围绕日志异常检测展开研究，主要基于 RNN 和状态抽象技术，探讨了提高日志异常检测模型可解释性的方法。首先，本文综述了现有的日志异常检测框架，涵盖日志收集、日志解析、特征提取和异常检测等环节。针对深度学习模型的黑盒性，本文章引入状态抽象技术，通过构建抽象模型提升检测过程的可解释性。最后，本文章展开实验验证，并通过具体案例说明如何解释模型，验证了整个毕业设计实现的工具的可用性。

本毕业设计的具体工作是实现了一个日志异常检测的可解释性工具，包括三个部分：一个用于训练日志异常检测模型的工具，允许用户自定义神经网络结构和输入日志序列形式；一个基于状态抽象的模型解释工具，通过聚类方法对 RNN 的隐向量进行抽象，生成抽象状态和状态轨迹；一个可视化工具，以图表和状态图的形式展示抽象模型和异常检测结果，并提供一定的交互性。

未来的研究可以加强实验验证，例如使用更多样的数据集，并与现有的基准方法进行比较。另一方面，未来的研究中可以尝试改进基于状态抽象的解释技术，并丰富可视化系统的视图，让解释工具更好地贴合日志异常检测任务。

参考文献

- [1] He S, Zhu J, He P, et al. Experience report: System log analysis for anomaly detection[C]//2016 IEEE 27th international symposium on software reliability engineering (ISSRE). IEEE, 2016: 207-218.
- [2] Chen Z, Liu J, Gu W, et al. Experience report: Deep learning-based system log analysis for anomaly detection[J]. arXiv preprint arXiv:2107.05908, 2021.
- [3] Lin Q, Zhang H, Lou J G, et al. Log clustering based problem identification for online service systems[C]//Proceedings of the 38th International Conference on Software Engineering Companion. 2016: 102-111.
- [4] Xu W, Huang L, Fox A, et al. Detecting large-scale system problems by mining console logs[C]//Proceedings of the ACM SIGOPS 22nd symposium on Operating systems principles. 2009: 117-132.
- [5] He P, Zhu J, Zheng Z, et al. Drain: An online log parsing approach with fixed depth tree[C]//2017 IEEE international conference on web services (ICWS). IEEE, 2017: 33-40.
- [6] Xu J, Yang R, Huo Y, et al. DivLog: Log Parsing with Prompt Enhanced In-Context Learning[C]//Proceedings of the IEEE/ACM 46th International Conference on Software Engineering. 2024: 1-12.
- [7] Zhu J, He S, He P, et al. Loghub: A large collection of system log datasets for ai-driven log analytics[C]//2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE). IEEE, 2023: 355-366.
- [8] Zhu J, He S, Liu J, et al. Tools and benchmarks for automated log parsing[C]//2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP). IEEE, 2019: 121-130.
- [9] He P, Zhu J, He S, et al. An evaluation study on log parsing and its use in log mining[C]//2016 46th annual IEEE/IFIP international conference on dependable systems and networks (DSN). IEEE, 2016: 654-661.

- [10] Hamooni H, Debnath B, Xu J, et al. Logmine: Fast pattern recognition for log analytics[C]//Proceedings of the 25th ACM international on conference on information and knowledge management. 2016: 1573-1582.
- [11] Vaarandi R. A data clustering algorithm for mining patterns from event logs[C]//Proceedings of the 3rd IEEE Workshop on IP Operations & Management (IPOM 2003) (IEEE Cat. No. 03EX764). Ieee, 2003: 119-126.
- [12] Zhang X, Xu Y, Lin Q, et al. Robust log-based anomaly detection on unstable log data[C]//Proceedings of the 2019 27th ACM joint meeting on European software engineering conference and symposium on the foundations of software engineering. 2019: 807-817.
- [13] Chen M, Zheng A X, Lloyd J, et al. Failure diagnosis using decision trees[C]//International Conference on Autonomic Computing, 2004. Proceedings. IEEE, 2004: 36-43.
- [14] Han J, Pei J, Tong H. Data mining: concepts and techniques[M]. Morgan kaufmann, 2022.
- [15] Liang Y, Zhang Y, Xiong H, et al. Failure prediction in ibm bluegene/l event logs[C]//Seventh IEEE International Conference on Data Mining (ICDM 2007). IEEE, 2007: 583-588.
- [16] Lou J G, Fu Q, Yang S, et al. Mining invariants from console logs for system problem detection[C]//2010 USENIX Annual Technical Conference (USENIX ATC 10). 2010.
- [17] Du M, Li F, Zheng G, et al. Deeplog: Anomaly detection and diagnosis from system logs through deep learning[C]//Proceedings of the 2017 ACM SIGSAC conference on computer and communications security. 2017: 1285-1298.
- [18] Du X, Xie X, Li Y, et al. Deepstellar: Model-based quantitative analysis of stateful deep learning systems[C]//Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. 2019: 477-487.
- [19] Wang Z, Huang Y, Song D, et al. Deepseer: Interactive rnn explanation and debugging via state abstraction[C]//Proceedings

- of the 2023 CHI Conference on Human Factors in Computing Systems. 2023: 1-20.
- [20] Li W, Zhang Z, Wang X, et al. Adax: Adaptive gradient descent with exponential long term memory[J]. arXiv preprint arXiv:2004.09740, 2020.
- [21] Dong G, Wang J, Sun J, et al. Towards interpreting recurrent neural networks through probabilistic abstraction[C]//Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering. 2020: 499-510.
- [22] Meng W, Liu Y, Zhu Y, et al. Loganomaly: Unsupervised detection of sequential and quantitative anomalies in unstructured logs[C]//IJCAI. 2019, 19(7): 4739-4745.
- [23] Nguyen K A, Walde S S, Vu N T. Integrating distributional lexical contrast into word embeddings for antonym-synonym distinction[J]. arXiv preprint arXiv:1605.07766, 2016.
- [24] Nedelkoski S, Bogatinovski J, Acker A, et al. Self-attentive classification-based anomaly detection in unstructured logs[C]//2020 IEEE International Conference on Data Mining (ICDM). IEEE, 2020: 1196-1201.
- [25] Farzad A, Gulliver T A. Unsupervised log message anomaly detection[J]. ICT Express, 2020, 6(3): 229-237.
- [26] Lu S, Wei X, Li Y, et al. Detecting anomaly in big data system logs using convolutional neural network[C]//2018 IEEE 16th Intl Conf on Dependable, Autonomic and Secure Computing, 16th Intl Conf on Pervasive Intelligence and Computing, 4th Intl Conf on Big Data Intelligence and Computing and Cyber Science and Technology Congress (DASC/PiCom/DataCom/CyberSciTech). IEEE, 2018: 151-158.

致谢

本毕业设计的完成离不开老师和学长的帮助,在此我要向所有帮助过我的人表示感谢。首先,我要感谢我的指导老师彭鑫老师,他给我的实验和论文提出了宝贵的建议。其次我要感谢张晨曦学长,他给我提供了选题上的诸多文献和资料,帮助我顺利开展实验。然后我要感谢王浩睿学长,他在实验的过程中为我答疑解惑,给了我不少启发。我还要感谢我的同学们,在他们的陪伴下,我在大学四年中不断成长。最后,我要感谢我的父母,他们总是给予我充分的理解和鼓励。